

Scenario 3 - File Upload

Prompt without Framework:

Weak Prompt: Generate test cases for file upload

Response Received:

Absolutely. For **file upload functionality**, I would test it across **positive, negative, boundary, security, UI, API, and usability scenarios**.

File Upload – Test Cases

TC ID	Test Scenario	Test Data / Condition	Expected Result	Priority
FU-001	Upload a valid file	Supported file, e.g., PDF	File uploads successfully	Critical
FU-002	Upload each supported file type	PDF, JPG, PNG, DOCX, XLSX etc.	All supported formats upload successfully	High
FU-003	Upload unsupported file type	.exe, .bat, .js etc.	Upload is rejected with appropriate message	Critical
FU-004	Upload file with minimum allowed size	File at minimum size limit	Upload succeeds	High
FU-005	Upload file exactly at maximum size	File = configured maximum size	Upload succeeds	Critical
FU-006	Upload file exceeding maximum size	File > configured limit	Upload is rejected with clear error	Critical
FU-007	Upload empty/0-byte file	0 KB file	Upload is rejected or handled according to requirements	High
FU-008	Upload file with long filename	Very long filename	System handles it without UI/API failure	Medium
FU-009	Upload file with special characters	test@#\$.pdf	File is handled according to filename rules	Medium
FU-010	Upload file containing spaces	My Test File.pdf	Upload succeeds	Medium
FU-011	Upload duplicate file	Same file uploaded twice	System handles duplicate according to requirement	Medium
FU-012	Upload multiple files	Multiple valid files	All files upload successfully if supported	High
FU-013	Upload maximum permitted number of files	Configured maximum count	Upload succeeds	High
FU-014	Upload more than permitted file count	Maximum + 1	Extra file is rejected with proper message	High
FU-015	Cancel file selection	Select file → Cancel	No file is uploaded/attached	Medium
FU-016	Remove uploaded file	Upload → Remove	File is removed successfully	High

TC ID	Test Scenario	Test Data / Condition	Expected Result	Priority
FU-017	Replace uploaded file	Upload A → Upload B	Correct file is retained according to requirement	Medium
FU-018	Upload same file with different extension	Rename extension without changing content	System validates actual file type and rejects if invalid	Critical
FU-019	Upload corrupted file	Corrupted PDF/image/document	File is rejected or handled gracefully	High
FU-020	Upload password-protected file	Password-protected PDF/ZIP	Behavior matches requirements	Medium
FU-021	Upload file with malicious content	Malware/test security payload	File is blocked/quarantined by security controls	Critical
FU-022	Upload executable disguised as document	file.pdf.exe	Upload is rejected	Critical
FU-023	Upload HTML/SVG containing scripts	Malicious content	File is rejected or safely sanitized	Critical
FU-024	Upload file with Unicode filename	résumé_日本語.pdf	File uploads and displays correctly	Medium
FU-025	Upload file with uppercase extension	TEST.PDF	Accepted if extension is supported	Medium
FU-026	Drag and drop valid file	Drag supported file to upload area	File uploads successfully	Medium
FU-027	Drag and drop unsupported file	Unsupported format	File is rejected	Medium
FU-028	Upload while network is disconnected	Disconnect network during upload	Proper error displayed; application doesn't crash	Critical
FU-029	Interrupt upload	Cancel/close during upload	Upload is stopped and system remains stable	High
FU-030	Upload large file over slow network	Simulated slow connection	Progress indicator works and upload eventually completes/fails gracefully	High
FU-031	Upload same file simultaneously	Multiple upload actions	No duplicate/corrupt records created	High
FU-032	Refresh page after upload	Upload → Refresh	File persistence matches requirement	High
FU-033	Navigate away during upload	Start upload → Navigate away	Appropriate warning/cancellation behavior	Medium
FU-034	Session timeout during upload	Session expires during upload	Upload fails safely and user receives appropriate message	High
FU-035	Upload without mandatory attachment	Leave file empty → Submit	Validation message displayed	Critical
FU-036	Download uploaded file	Upload → Download	Correct file downloads successfully	High
FU-037	Verify uploaded file content	Upload known file → Open/download	File content remains unchanged	Critical
FU-038	Verify filename after upload	Upload file	Original/expected filename displayed	Medium

TC ID	Test Scenario	Test Data / Condition	Expected Result	Priority
FU-039	Verify file size after upload	Upload known-size file	Displayed/stored size is correct	Medium
FU-040	Verify file metadata	Upload file	Required metadata is stored correctly	Medium

Important Boundary Tests

These are especially important for QA:

Test	Boundary
File size	0 KB, 1 byte, minimum-1, minimum, maximum-1, maximum, maximum+1
File count	0, 1, maximum-1, maximum, maximum+1
Filename	Empty, 1 character, maximum length, maximum+1
Extension	Valid lowercase, uppercase, mixed case, missing extension, multiple extensions
Network	Fast, slow, intermittent, disconnected
Concurrent upload	1, maximum supported concurrent uploads, maximum+1

Security Test Cases — Critical

1. Upload executable files such as .exe, .bat, .cmd.
2. Upload files containing malicious scripts.
3. Attempt **double-extension bypass**, e.g. invoice.pdf.exe.
4. Attempt MIME-type manipulation.
5. Change Content-Type while sending the upload request through API.
6. Upload oversized files to test DoS/resource-exhaustion protection.
7. Attempt path traversal through filename, such as ../../test.pdf.
8. Verify uploaded files cannot be accessed by unauthorized users.
9. Verify one user's uploaded file cannot be downloaded using another user's ID/reference.
10. Verify authentication/authorization is enforced on upload APIs.
11. Verify files are scanned for malware where applicable.
12. Verify sensitive files aren't exposed through predictable URLs.
13. Verify uploaded files cannot execute as server-side scripts.
14. Verify appropriate file permissions are applied to stored files.
15. Verify deleted files are no longer accessible through old download links.

API-Level Validation

For the upload API, validate:

- HTTP status code
- Response time
- Response body/schema
- File ID/reference generation
- Filename
- File size
- MIME type

- Upload timestamp
- Authentication
- Authorization
- Duplicate handling
- Invalid file handling
- Maximum file-size validation
- Maximum file-count validation
- Error response/message
- Server-side file-type validation
- Virus/malware scanning response
- Storage failure handling
- Database/storage consistency

End-to-End Critical Flow

A high-priority regression scenario would be:

Select file → Validate extension → Validate size → Upload → Progress → Server validation → Virus scan → Store file → Save metadata → Display attachment → Download → Verify downloaded file → Delete → Verify file is no longer accessible.

This flow catches defects that a simple **"file uploaded successfully"** test would miss.

Prompt with Framework:

Provide test cases as word document to Validate the file upload feature in case creation module, as a functional tester I would like to validate the case creation functionality by uploading a file which is mandatory step to complete flow and the file size allowed is 10mb and do not accept xml and Json files .

Response Received:

Done. I created a **professional Word document** containing functional test cases specifically for the **Case Creation → Mandatory File Upload** flow, covering:

- Valid uploads
- Mandatory attachment validation
- **10 MB boundary testing**
- Files >10 MB
- XML rejection
- JSON rejection
- Invalid → valid file recovery
- Remove/replace attachment
- Persistence after case creation
- File integrity
- Validation messages
- Negative and critical regression scenarios
- Acceptance criteria

Observation:

1. Prompt provided using framework provides a structured Output

2. Scope of Validation is improved because of defined context
3. With framework, User shall control LLM from providing default outputs and very generic test cases